SAUCEDEMO

WEB APPLICATION QUALITY ASSURANCE TEST PLAN

Manual Functional Testing Portfolio Project

Document ID	Version	Status
QA-TP-001	1.0	Final Portfolio Baseline
Prepared By	Role	Application Under Test
Nancy McKenzie	Quality Assurance Analyst	SauceDemo
Test Approach	Primary Browser	Test Management
Manual Functional Testing	Google Chrome	Microsoft Excel / GitHub

PORTFOLIO USE NOTICE

This document is a professional QA portfolio artifact based on the public SauceDemo demonstration application. It does not represent employment by, sponsorship from, or an internal test plan of Sauce Labs.

TABLE OF CONTENTS

SauceDemo Web Application Quality Assurance Test Plan

1. Document Control

This section establishes ownership, version control, intended use, and governance for the SauceDemo QA Test Plan. The document serves as the controlling reference for the scope, strategy, execution standards, defect handling, and completion criteria used throughout this portfolio project.

Field	Value
Document Title	SauceDemo Web Application Quality Assurance
Document ID	QA-TP-001
Version	1.0
Prepared By	Nancy McKenzie
Role	Quality Assurance Analyst
Application Under Test	SauceDemo - E-Commerce Online Retail Demo
Testing Method	Manual Functional Testing
Project Model	Agile-inspired, risk-based portfolio simulation
Repository	QA-Manual-Testing-ShopSmart
Classification	Professional Portfolio / Educational Use

1.1 Revision History

Version	Change Summary	Owner	Status
0.1	Initial test plan	Nancy McKenzie	Draft
0.5	Test strategy, controls	Nancy McKenzie	Reviewed
1.0	Final portfolio	Nancy McKenzie	Final

1.2 Approval and Sign-Off

Because this is an individual portfolio project, formal organizational approvals are simulated. The approval structure below demonstrates how ownership and sign-off would normally be recorded in a QA engagement.

Role	Name / Owner	Approval Responsibility
QA Analyst / Test Owner	Nancy McKenzie	Test planning, execution
Business/Product	Simulated Stakeholder	Business-rule clarification and
Development Representative	Simulated Development Team	Defect investigation
Release Decision Owner	Simulated Product Owner	Final release decision based on

1.3 Document Usage

Use this Test Plan as the source of truth for the project testing scope and execution approach.

Trace test scenarios and test cases back to the business requirements defined for the project.

Record actual execution evidence and defects in the corresponding portfolio artifacts rather than modifying planned expectations retroactively.

Update version history when scope, strategy, entry/exit criteria, or material risks change.

2. Executive Summary

The SauceDemo QA project evaluates the core customer journey of a demonstration e-commerce application through structured manual testing. The focus is not simply to prove that pages load; it is to determine whether the application behaves consistently across authentication, inventory, product selection, cart management, checkout, order completion, navigation, and validation scenarios.

The test approach is risk-based. Highest attention is given to workflows that directly affect a customer's ability to access the store, select products, maintain an accurate cart, and complete an order. Negative and boundary-oriented scenarios are included where the user can submit invalid, incomplete, or unexpected input. Regression coverage is used after defects are considered resolved to confirm that corrections do not disrupt previously working behavior.

This Test Plan intentionally distinguishes between planned work and observed results. Execution counts, pass/fail totals, defect severity totals, and final release conclusions belong in the Test Execution Report and Test Summary Report after testing is performed. This prevents fabricated metrics from being presented as actual evidence and keeps the portfolio defensible during interviews.

2.1 Quality Goals

Protect the critical purchase path from login through order confirmation.

Verify that application state, product information, and cart contents remain accurate as users navigate the site.

Confirm required-field and authentication validation produces clear, appropriate feedback.

Identify reproducible defects with sufficient evidence for efficient investigation and retesting.

Maintain traceability from business requirements to test scenarios, test cases, execution results, and defects.

2.2 Success Definition

Testing is considered successful when the agreed in-scope requirements have appropriate test coverage, planned critical-path cases have been executed, material defects have been documented and dispositioned, and the remaining risk is understood well enough to support a reasoned release recommendation. A high pass percentage alone is not treated as sufficient evidence of quality.

3. Introduction

3.1 Purpose

The purpose of this Test Plan is to define the testing objectives, boundaries, responsibilities, methods, controls, and quality criteria for manual functional testing of the SauceDemo web application. It establishes a repeatable framework for designing and executing tests and for communicating quality risk in a way that is understandable to technical and non-technical stakeholders.

The document also serves as a portfolio demonstration of practical QA planning. It is written to show how a QA Analyst translates an application's business workflows into a controlled testing effort rather than treating test cases as isolated checklists.

3.2 Intended Audience

QA Analysts and Testers reviewing the planned approach and execution standards.

Developers reviewing expected defect information and retest expectations.

Product Owners or Business Analysts reviewing functional coverage and business risk.

Hiring managers or interviewers evaluating the author's understanding of QA planning and STLC practices.

3.3 Reference Application

Application Under Test: SauceDemoPublic URL: https://www.saucedemo.com/Application Category: Demonstration e-commerce / online retail web applicationPrimary Testing Interface: Web browser

3.4 Test Planning Principles

Risk first: prioritize customer-critical and revenue-critical workflows before cosmetic behavior.

Traceability: every planned test should have a clear reason for existing and map to a requirement or identified risk.

Reproducibility: defects must contain enough information for another person to reproduce the observed behavior.

Evidence over assumption: planned metrics are not reported as actual results until execution evidence exists.

Independence of artifacts: requirements define expected behavior; test cases validate it; execution reports record results; defect records document deviations.

4. Project Overview

SauceDemo provides a controlled retail-style environment for practicing software testing. The application presents an authenticated product inventory and supports a customer flow that includes selecting products, managing a cart, entering checkout information, reviewing an order, and completing the transaction. It also exposes multiple test-user profiles designed to demonstrate different application behaviors.

4.1 Business Context

For this portfolio engagement, the application is treated as an online retail storefront. The simulated business objective is to provide a reliable purchase experience in which an authorized customer can enter the store, identify products, build an accurate cart, and complete checkout without unexpected functional interruption or misleading information.

4.2 Core Customer Journey

1. Authenticate with a valid user account.
2. Review available inventory and product information.
3. Sort or inspect products to support a purchase decision.
4. Add selected products to the shopping cart.
5. Modify or remove cart selections when needed.
6. Proceed to checkout and enter required customer information.
7. Review order details and complete the purchase.
8. Receive clear confirmation that the order flow completed.
9. Log out or continue navigating without corrupting application state.

4.3 Primary Functional Areas

Functional Area	Quality Focus
Authentication	Valid/invalid login behavior, required fields
Inventory	Product availability, names, descriptions
Product Detail	Correct item information and navigation
Shopping Cart	Add/remove behavior, item identity
Checkout Information	Required fields, accepted input, validation
Checkout Overview	Order contents, item pricing, totals, navigation
Order Completion	Successful completion state and confirmation
Menu / Session	Navigation options, logout behavior, reset-

4.4 Project Constraints

The project uses a public demonstration environment; source code, backend services, deployment controls, and production telemetry are not available to the tester.

The project is manual-testing focused; API, database, performance, penetration, and automated UI testing are excluded from this project and may be demonstrated separately in other portfolio work.

Observed application behavior may change if the public demo application is updated by its owner.

No claim is made that this portfolio represents internal Sauce Labs requirements, release criteria, or defect history.

5. Business and Test Objectives

5.1 Simulated Business Objectives

ID	Business Objective
BO-01	Customers can reliably access the storefront
BO-02	Customers can review product information
BO-03	Customers can add, retain, and remove the
BO-04	Customers can progress through checkout with
BO-05	Customers receive a clear completion state
BO-06	Navigation and session behavior do not

5.2 Test Objectives

ID	Test Objective	Evidence
TO-01	Validate positive and negative	Executed login test cases and
TO-02	Verify inventory and product-	Inventory/product test cases
TO-03	Verify cart actions preserve the	Cart test cases, execution
TO-04	Validate required checkout	Checkout test cases and
TO-05	Verify order overview and	Overview/completion test
TO-06	Assess navigation, logout, and	Navigation/session cases and
TO-07	Document deviations with	Defect Log and individual Bug
TO-08	Maintain requirement-to-test	RTM and Test Summary Report

5.3 Quality Attributes Addressed

Functional correctness - expected business actions produce the intended result.

Consistency - similar actions and application states behave predictably.

Usability signals - messages and navigation provide enough clarity for a customer to continue or correct an error.

Data presentation integrity - product identity, pricing, and order information remain coherent within the visible user flow.

Recoverability within the UI - users can correct invalid entries, navigate back when appropriate, or safely exit the session.

5.4 Non-Objectives

This project does not certify SauceDemo for production use and does not attempt to validate regulatory compliance, payment-card security, production-scale reliability, backend data integrity, accessibility conformance, or security posture. Those areas require different access, tools, expertise, and evidence than are available in this manual portfolio exercise.

6. Stakeholders and Responsibilities

Detailed execution responsibilities are defined later in the complete Test Plan. The planning-level view below establishes who would consume or act on the outputs in a typical software team.

Stakeholder	Primary Interest	Expected Interaction
QA Analyst	Coverage, execution quality	Owns test artifacts and
Product Owner / Business	Business behavior and	Clarifies expected behavior
Developer	Technical diagnosis and	Reviews defects, implements
Release Owner	Go/no-go decision	Uses QA evidence and business
Portfolio Reviewer / Hiring	Demonstrated QA competency	Evaluates planning

7. Test Scope

The test scope establishes the functional boundaries of the SauceDemo manual testing effort. Scope is intentionally based on functionality observable through the public web interface. The project does not imply access to Sauce Labs source code, internal requirements, databases, services, production logs, or release processes.

7.1 In-Scope Functional Areas

Functional Area	Planned Coverage	Risk Priority
Authentication	Valid login, invalid login, locked user	Critical
Inventory / Products	Inventory page load, product names	High
Product Sorting	Name sorting A-Z / Z-A and price	Medium
Shopping Cart	Add item, remove item, multiple item	Critical
Checkout - Information	Checkout initiation, first name, last	Critical
Checkout - Overview	Selected items, quantities, item prices	Critical
Order Completion	Completion confirmation, post-order	Critical
Navigation / Menu	Menu access, inventory navigation	High
Error Handling / Validation	User visible feedback for invalid	High
Session / State	Visible persistence or reset of cart and	High

7.2 Out-of-Scope Activities

API and service-layer testing, including direct endpoint validation.

Database testing, SQL validation, data persistence verification outside what is visible in the UI.

Performance, load, stress, endurance, scalability, or capacity testing.

Penetration testing, vulnerability scanning, authentication bypass attempts, or other security testing.

Formal accessibility conformance testing against WCAG or Section 508.

Automated UI testing and test automation framework development.

Native mobile application testing; SauceDemo is treated as a browser-based web application in this project.

Formal cross-browser certification. Chrome is the primary execution browser; other browsers are not part of committed coverage unless documented as exploratory observations.

Backend payment processing, real payment-card authorization, shipping fulfillment, inventory synchronization, tax engine validation, email/SMS notifications, or other services not exposed by the demo application.

Production monitoring, production deployment validation, or operational support.

7.3 Scope Control

Any newly observed feature may be explored, but it is not automatically added to committed scope. A material scope change should be documented in revision history and evaluated for impact on test cases, schedule, traceability, and exit criteria. This prevents silent scope expansion and protects the credibility of reported coverage.

8. Test Strategy and Overall Approach

The strategy combines requirements-based, risk-based, workflow-based, and exploratory testing. The project prioritizes customer-critical flows while maintaining traceability to documented requirements. Testing is performed manually from the user interface, with actual outcomes recorded separately from planned expectations.

8.1 Risk-Based Prioritization

Priority	Definition	Examples	Execution Expectation
P1 – Critical	Failure prevents or	Valid login; cart accuracy;	Execute first in
P2 – High	Failure significantly impairs	Product detail navigation;	Execute during primary
P3 – Medium	Failure affects secondary	Sort order; navigation	Execute after P1/P2
P4 – Low	Minor cosmetic or low	Spacing, minor visual	Document if observed;

8.2 Test Levels

This portfolio project operates primarily at the system / end-to-end user-interface level. Unit and component testing are assumed to be development responsibilities and are not observable from the public application. Integration behavior is evaluated only where it is visible through the complete user workflow, such as cart-to-checkout progression.

8.3 Planned Test Types

Test Type	Purpose	Application to SauceDemo
Smoke Testing	Determine whether the	Validate login, inventory access, add
Functional Testing	Verify features perform expected	Validate each in-scope requirement
Regression Testing	Confirm changes or defect corrections	Re-execute affected tests plus critical
Exploratory Testing	Investigate behavior beyond scripted	Explore navigation, unusual
Positive Testing	Confirm valid inputs/actions produce	Valid credentials, valid checkout data
Negative Testing	Confirm invalid or missing inputs are	Invalid credentials, blank fields
UI / Usability Observation	Assess visible consistency and clarity	Labels, error clarity, navigation cues
Retesting	Verify a reported defect is corrected	Repeat original reproduction steps

9. Test Design Techniques

Test cases will be designed to provide meaningful coverage without inflating the portfolio with repetitive cases. The following techniques are selected because they map naturally to SauceDemo's visible workflows and are commonly used in manual QA.

Technique	How It Is Used	Example
Requirements-Based Testing	Derive at least one test condition from	Requirement for valid login maps to
Equivalence Partitioning	Group inputs or user states expected	Valid credentials vs invalid
Boundary / Required-Field Testing	Test transitions around required or	Blank username/password; missing
Decision / Condition Testing	Vary meaningful conditions that	User account type and credential
State Transition Testing	Validate behavior as the user moves	Inventory -> cart -> checkout
End-to-End Scenario Testing	Validate complete customer journeys	Login, select products, verify cart
Error Guessing	Use QA experience to target likely	Repeated add/remove actions, back
Exploratory Testing	Use time-boxed investigation guided	Explore cart and menu state while

9.1 Test Case Design Standard

Each formal test case should contain enough information for another tester to execute it consistently. At minimum, the test case workbook will capture: Test Case ID, Requirement ID, module, title/objective, priority, preconditions, test data, numbered steps, expected result, actual result, status, defect reference (when applicable), execution date, and notes/evidence reference.

9.2 Expected Result Quality

Expected results will describe observable behavior rather than vague statements such as "works correctly." For example, an authentication case should state the expected destination or error message behavior; a cart case should state the expected item identity, quantity, badge state, and visible price information where relevant.

10. Test Environment and Configuration

The environment is intentionally simple and reproducible because the application is a public demonstration site. Environment details must be recorded during execution so a defect report distinguishes application behavior from local configuration issues.

Environment Item	Planned Configuration	Control / Evidence
Application	SauceDemo public web application	https://www.saucedemo.com/
Environment Classification	Public demo / portfolio test	Not represented as an internal Sauce
Primary Browser	Google Chrome, current stable	Record exact browser version for
Operating System	Windows 11 or macOS, based on	Record actual OS used in execution
Screen / Device	Desktop or laptop browser viewport	Record viewport only when layout
Network	Standard consumer internet	No performance conclusions will be
Test Management	Microsoft Excel / GitHub portfolio	Controlled IDs and consistent status
Defect Tracking	Structured Excel defect log plus	Defect IDs linked to failed test cases
Evidence	Screenshots and notes stored in	Avoid exposing unrelated personal

10.1 Browser and Cache Control

Begin formal execution from a known state when practical, including a fresh authenticated session for tests that require one.

If cached state or browser history may affect a result, repeat the test in a clean session before logging a defect.

Do not clear state during a test if persistence itself is the behavior being evaluated.

Record browser version and relevant session conditions for reproducible defects.

10.2 Environment Limitations

Because the tester does not control deployment or backend configuration, environment outages, application updates, or externally introduced behavior may occur without notice. If the site is unavailable or materially changed, execution should be paused for affected cases and the condition documented rather than incorrectly marking tests as failed.

11. Test Data Strategy

SauceDemo publishes test credentials on its login page for demonstration use. Test data will use only credentials and values appropriate for the public demo. No real customer information, passwords, payment information, or personally sensitive data will be entered or stored in portfolio artifacts.

11.1 User Profiles

Profile Category	Purpose in Testing	Handling
Standard / expected behavior user	Baseline functional testing of the	Use published demo credentials; do
Locked user	Validate authentication rejection and	Use only the demo profile
Problem / behavior variant users	Support exploratory and negative	Document observed behavior without
Other published demo users	Used only when a specific test	Record profile used in the test

11.2 Checkout Data

Checkout fields will use clearly fictitious, non-sensitive values. Positive tests will provide syntactically reasonable first name, last name, and postal code values. Negative tests will omit required values or vary input only within normal UI use. The project will not use real addresses or attempt injection/security payloads.

11.3 Data Reset and Independence

Where possible, each test case should establish its own preconditions rather than depend on the result of a previous case.

Cart state should be cleared or deliberately established before cases that validate cart contents or badge counts.

Tests that intentionally depend on a prior workflow step should be identified as end-to-end scenarios rather than accidentally chained cases.

If the application offers a reset-state function, use it only when consistent with the test objective and document its use when it affects results.

12. Coverage Model and Traceability

Coverage will be demonstrated through a Requirements Traceability Matrix (RTM), not through an unsupported claim that “everything was tested.” Each requirement will map to one or more test scenarios and test cases. Execution status and defect references will later be added so the RTM shows both planned and achieved coverage.

Coverage Layer	Identifier Example	Purpose
Business Requirement	BR-001	Defines expected business behavior
Test Scenario	TS-001	Defines a high-level condition or
Test Case	TC-AUTH-001	Defines repeatable execution steps
Execution Result	PASS / FAIL / BLOCKED / NOT RUN	Records what actually happened
Defect	BUG-001	Documents a reproducible deviation
Evidence	Screenshot / note reference	Supports the execution or defect

12.1 Coverage Targets

100% of documented in-scope business requirements mapped to at least one test scenario and one executable test case before closure.

All P1/Critical requirements represented in smoke or critical-path coverage as appropriate.

Positive and negative coverage for input-validation requirements where both valid and invalid behavior are meaningful.

Regression coverage selected based on defect impact and shared workflow risk rather than blindly re-running every case.

Any requirement not executed or not fully testable must be explicitly identified as residual risk in the Test Summary Report.

12.2 Coverage Does Not Equal Quality

A requirement can have test coverage and still contain risk. Traceability proves that a condition was considered and tested; it does not prove absence of defects. The final quality assessment will consider defect severity, critical-path behavior, blocked or unexecuted coverage, reproducibility, and remaining uncertainty in addition to pass/fail counts.

13. Exploratory Testing Strategy

Scripted test cases provide repeatability, while exploratory testing provides adaptability. Exploratory sessions will be time-boxed and charter-driven so they add evidence rather than becoming undocumented clicking.

Exploratory Charter	Focus	Example Questions
EXP-01 Authentication & Session	Login variations, error recovery	Does the application clearly recover
EXP-02 Inventory & Product	Product identity, detail pages, sorting	Do names/prices remain consistent?
EXP-03 Cart State	Add/remove sequences, multiple	Can visible cart state become
EXP-04 Checkout Recovery	Cancel/back behavior, missing	Can the user recover from an
EXP-05 Variant Demo Users	Behavior of published non-standard	Is the behavior intentional to the

13.1 Exploratory Evidence

Each session should record the charter, date, approximate duration, user profile, areas explored, notable observations, defects created, and follow-up tests. Exploratory findings that become repeatable regression needs should be converted into formal test cases.

14. Regression and Retest Strategy

Retesting and regression are related but distinct. Retesting confirms that a specific reported defect is corrected. Regression testing checks whether the correction or other change adversely affected related or critical functionality.

14.1 Retest Procedure

10. Review the defect record and confirm the reported correction or changed behavior is available for testing.
11. Recreate the original environment, user profile, preconditions, and test data as closely as practical.
12. Repeat the documented reproduction steps.
13. Compare the actual result with the expected result and acceptance condition.
14. Record PASS/FAIL for retest and attach updated evidence where useful.
15. If the issue remains, reopen or update the defect with current evidence rather than creating a duplicate unless the behavior is materially different.

14.2 Regression Selection

Change / Defect Area	Minimum Regression Consideration
Authentication	Valid login, invalid login, inventory landing, logout/session
Inventory / Product	Inventory rendering, affected product details, sorting if
Cart	Add/remove, badge, multi-item contents, checkout entry
Checkout Information	Required field validation, valid progression, cancel
Checkout Overview / Completion	Selected items, totals presentation, finish action
Navigation / Menu	Critical destination links, logout and state behavior related

15. Test Status and Evidence Standards

Consistent status definitions prevent misleading metrics. The execution workbook and Test Summary Report will use the following controlled vocabulary.

Status	Definition	Use Rule
PASS	Actual behavior matches the	Use only after the case is executed
FAIL	Actual behavior materially differs	Link a defect or documented issue
BLOCKED	Execution cannot be completed	State the blocker; do not count as a
NOT RUN	The test has not been executed in the	Remain transparent; do not convert to
N/A	The test is formally determined not	Requires a reason; avoid using as a

15.1 Evidence Expectations

Capture screenshots for failed cases when visual evidence materially helps reproduce or understand the issue.

Use descriptive filenames that reference the test case or defect ID.

Do not overload the repository with screenshots of every successful click; evidence should be purposeful.

Keep expected result, actual result, and defect evidence consistent across the test case, defect log, and summary artifacts.

Remove or crop unrelated personal information before publishing evidence to GitHub.

16. Entry, Exit, Suspension and Resumption Criteria

Formal criteria prevent testing from becoming an open-ended activity. They establish when meaningful execution can begin, when testing may be considered complete, and when conditions are poor enough that execution should stop rather than produce unreliable results.

16.1 Entry Criteria

Entry Criterion	Required Condition
Application availability	SauceDemo is reachable in the planned
Scope baseline	In-scope functional areas and business
Test cases	Critical and high-priority test cases are
Test data	Required SauceDemo test users and non-
Environment	Browser, operating system, connectivity, and
Defect process	Defect identifiers, severity/priority definitions
Traceability	Requirements and test cases use stable

16.2 Exit Criteria

All planned Critical-priority test cases have been executed unless a documented blocker prevents execution.

High-priority coverage is substantially complete and any unexecuted cases have a documented reason and risk assessment.

All observed defects are recorded with sufficient reproduction information and a current disposition.

Critical and High severity defects are either resolved and retested successfully or explicitly documented as accepted residual risk for the portfolio simulation.

Relevant regression tests have been executed after material corrections or state-changing issues.

The RTM reflects final coverage and links requirements to test cases, execution status, and defects where applicable.

A Test Summary Report communicates execution results, unresolved risks, limitations, and the QA release recommendation.

16.3 Suspension Criteria

The public SauceDemo environment is unavailable or unstable enough that results cannot be distinguished from environment failure.

A blocker prevents access to a critical functional area required for the planned test cycle.

Unexpected application changes invalidate a material portion of the current test cases or expected results.

Test data or account behavior becomes unreliable and could create misleading execution evidence.

16.4 Resumption Criteria

Testing resumes after the blocking condition is removed, the affected environment or account is revalidated with a smoke check, and any impacted test cases are reviewed for continued applicability. Tests affected by the interruption are re-executed when confidence in the earlier result is reduced.

17. Defect Management Process

Defect management is designed to make findings reproducible, triageable, and traceable. A defect is not considered strong evidence merely because something looked wrong; the observation must be repeatable or clearly documented as intermittent, tied to an expected behavior, and supported by appropriate evidence.

17.1 Defect Lifecycle

New - tester records the observed deviation and supporting evidence.

Reviewed - defect information is checked for completeness, reproducibility, severity, and priority.

Open - issue is accepted for investigation or remediation in the simulated workflow.

Resolved - a correction is considered available for verification.

Retest - QA repeats the original reproduction path using the corrected state.

Closed - expected behavior is confirmed and relevant regression coverage is satisfactory.

Reopened - the original issue persists, recurs, or the correction does not satisfy the expected result.

Deferred / Accepted Risk - issue remains unresolved but is explicitly documented as residual risk.

17.2 Required Defect Fields

Field	Required Content
Defect ID	Unique identifier such as BUG-001
Summary	Concise description of the observed problem
Requirement / Test Case	Related requirement and originating test case
Environment	Browser, OS, application URL/area, and relevant
Preconditions	State required before reproduction
Steps to Reproduce	Numbered, repeatable actions
Expected Result	Behavior required by the requirement or
Actual Result	Observed behavior without speculation
Severity	Impact classification using the project severity
Priority	Remediation urgency using the project priority

Field	Required Content
Evidence	Screenshot or other supporting artifact where
Status	Current lifecycle state
Retest Result	Pass/fail outcome after a correction is available

17.3 Severity Model

Severity	Definition	Example Impact
Critical	Prevents a critical business	Valid customer cannot
High	Major function is broken or	Cart or checkout behavior
Medium	Functional issue affects a non	Incorrect validation message
Low	Minor cosmetic, wording, or	Small alignment or

17.4 Priority Model

Priority	Meaning
P1 – Urgent	Address before release consideration because
P2 – High	Address promptly; issue materially affects
P3 – Normal	Plan correction based on sprint/release
P4 – Low	Correct when practical; low business impact

Severity and priority are intentionally separate. A technically severe defect may receive a different remediation priority depending on exposure, workaround, and release context. Conversely, a lower-severity issue may receive higher priority if it affects a highly visible or frequently used workflow.

18. Defect Triage, Retesting and Regression Control

18.1 Triage Principles

Confirm the issue is not expected behavior of a SauceDemo special test profile before logging it as a defect.

Separate environment instability, test-data problems, and tester error from application defects.

Use business impact and reproducibility to support severity rather than choosing severity based on how unusual a defect appears.

Avoid duplicate defects by checking the existing defect log before creating a new record.

Link related test cases so one defect affecting multiple scenarios is visible without inflating defect counts.

18.2 Retest Standard

A resolved defect is retested by repeating the original steps and preconditions as closely as possible. The tester confirms the specific correction, records the retest result, and preserves evidence when

the correction is important or visually demonstrable. A defect is reopened if the original failure remains or the stated correction does not satisfy the expected result.

18.3 Regression Selection

Regression scope is selected according to the functional area touched by the issue and the likelihood of collateral impact. Changes affecting login trigger authentication and session checks; cart-related corrections trigger add/remove/state and checkout-entry coverage; checkout corrections trigger downstream overview and completion checks. Critical purchase-path smoke tests are included after material changes.

18.4 Defect Closure Quality Check

16. Original expected behavior is now satisfied.
17. No obvious adjacent workflow was broken by the correction.
18. Retest evidence and status are recorded.
19. Related test execution records and RTM links are updated when applicable.

19. Roles and Responsibilities

This is an individual portfolio project, so Nancy McKenzie performs the hands-on QA activities. The table also shows how the same responsibilities would be distributed across a cross-functional software team.

Role	Responsibilities in This Project / Real Team
QA Analyst – Nancy McKenzie	Own test planning; analyze requirements;
Product Owner / Business Analyst – Simulated	Clarify expected business behavior, prioritize
Developer – Simulated	Investigate reproducible defects, provide
Release Owner – Simulated	Consider QA evidence, unresolved defects, and

19.1 QA Analyst Decision Responsibilities

Escalate blockers and high-impact defects promptly rather than waiting until test closure.

Challenge ambiguous expected results instead of manufacturing a pass/fail outcome.

Preserve independence between planned expectations and actual results.

Communicate limitations and residual risk even when overall execution appears favorable.

20. Communication and Status Reporting

Communication	Timing	Content
Test Readiness Check	Before execution	Scope, environment, test data

Communication	Timing	Content
Execution Status	During active testing	Cases executed
Defect Triage	As needed	Reproducibility, severity
Closure Summary	End of cycle	Coverage, results, defect

Because this portfolio does not operate within a live Scrum team, the communications are represented through the project artifacts. In a real Agile team, the same information would typically be surfaced through stand-ups, sprint ceremonies, defect triage, dashboards, and release-readiness discussions.

21. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
Public demo	Medium	High	Confirm availability
Application behavior	Medium	Medium	Review affected
Intentional special	Medium	High	Use standard_user as
Limited access to	High	Medium	Restrict conclusions to
Single browser focus	High	Medium	Explicitly classify
Portfolio metrics	Medium	High	Use stable IDs; derive
Over testing low risk	Medium	Medium	Use risk based
Ambiguous expected	Medium	Medium	Document assumption;
Evidence contains	Low	High	Use only demo

21.1 Residual Risk Philosophy

Testing reduces uncertainty; it does not prove the absence of defects. The final Test Summary will identify what was tested, what was not tested, unresolved defects, and important environmental limitations so that the release recommendation is based on transparent residual risk rather than an unsupported claim that the application is defect-free.

22. Test Schedule and Milestones

The project uses a two-week portfolio simulation. The schedule is intentionally compact but follows a realistic sequence in which requirements and risk drive test design, execution produces evidence, and closure follows reconciliation of results.

Period	Primary Activities	Outputs
Days 1-2	Requirement analysis	Business Requirements, initial
Days 3-4	Scenario design, test case	Test Scenarios, Test Cases
Day 5	Smoke validation and critical-	Initial execution results and
Days 6-7	Functional, negative, UI, and	Execution evidence and Defect
Day 8	Defect review, targeted	Retest records and regression
Day 9	Coverage reconciliation and	Updated RTM and outstanding-

Period	Primary Activities	Outputs
Day 10	Test closure and portfolio	Test Summary Report, final

22.1 Schedule Control

Critical-path execution takes precedence if time becomes constrained.

Blocked cases are not silently removed from scope; they remain visible with the reason documented.

Closure is delayed if project metrics cannot be reconciled across execution, defects, and traceability artifacts.

23. Test Deliverables

Artifact	Purpose	Planned Format
Test Plan	Defines scope, strategy	DOCX / PDF
Business Requirements	Defines portfolio level	DOCX / PDF
Test Scenarios	Provides high-level coverage of	XLSX
Test Cases	Defines detailed preconditions	XLSX
Test Execution Report	Records actual execution status	XLSX
Defect Log	Tracks all documented defects	XLSX
Individual Bug Reports	Provides detailed evidence for	DOCX / PDF
Requirements Traceability	Maps requirements to	XLSX
Test Summary Report	Communicates final coverage	DOCX / PDF
Screenshots / Evidence	Supports reproducibility and	PNG / JPG

24. Metrics and Quality Reporting

Metrics will be calculated from actual project artifacts after execution. The Test Plan defines what will be measured but does not pre-populate favorable results. This preserves evidence integrity and prevents planned numbers from being mistaken for completed testing.

Metric	Definition / Use
Test Design Coverage	Number of requirements mapped to at least
Execution Progress	Executed cases compared with planned
Pass / Fail / Blocked	Actual status distribution using the project
Defect Count by Severity	Observed defects grouped by impact
Defect Status	Open, resolved, retest, closed, reopened
Requirement Coverage	Requirement to test execution visibility
Critical Path Completion	Execution status of login to order completion
Residual Risk	Material unresolved issues, untested areas

24.1 Metric Interpretation Rules

A high pass rate does not override an unresolved Critical defect.

Blocked tests remain visible and are not counted as passes.

Duplicate defects are not counted as separate product issues merely to increase defect totals.

Coverage is evaluated against the agreed portfolio requirements, not against unknown internal SauceDemo requirements.

Final published counts must match across the Test Execution Report, Defect Log, RTM, README, and Test Summary Report.

25. Assumptions and Dependencies

25.1 Assumptions

SauceDemo remains publicly accessible during the planned execution period.

The visible application behavior is sufficient to define portfolio-level functional expectations for the selected workflows.

Standard demo credentials displayed or intended for the application may be used for testing purposes.

Fictional, non-sensitive values will be used for checkout information.

The standard_user profile serves as the primary baseline for normal customer behavior unless a test explicitly targets another profile.

25.2 Dependencies

Reliable internet connectivity and browser access.

Availability of Microsoft Excel/Word or compatible tools for documentation.

GitHub availability for publishing portfolio artifacts.

Stable requirement and test identifiers so traceability remains intact across documents.

25.3 Constraints

No access to SauceDemo source code, internal requirements, database, CI/CD pipeline, server logs, or production monitoring.

No organizational product owner or development team is available to formally adjudicate ambiguous behavior; assumptions must therefore be transparent.

The project is designed to demonstrate manual QA competency and does not claim full product certification.

26. Test Plan Change Control

Material changes to scope, strategy, quality gates, or expected behavior require a version update. Minor wording or formatting corrections may be incorporated without changing the execution baseline when they do not alter testing intent.

Change Type	Required Action
New in-scope feature	Add/update requirement, scenarios, test cases
Expected behavior clarification	Update requirement and impacted test cases;
Environment change	Record new configuration and determine
Test strategy change	Update Test Plan version and explain the
Defect correction	Do not rewrite the original expectation;

27. Test Closure and Release Recommendation Framework

At test closure, QA will not issue a simple “passed” statement. The Test Summary Report will provide a recommendation based on coverage, critical-path execution, defect disposition, blockers, untested scope, and residual risk.

27.1 Recommendation Categories

Recommendation	Meaning
Recommend Release	Critical path objectives are met and remaining
Recommend Release with Known Risk	Core objectives are met, but unresolved non
Do Not Recommend Release	Critical functionality is blocked or materially

27.2 Closure Checklist

Reconcile test-case totals and execution statuses.

Reconcile defect counts and lifecycle statuses.

Confirm RTM coverage and defect links.

Verify critical and high-severity defect dispositions.

Document untested or blocked scope.

Summarize environmental and methodological limitations.

State residual risk in plain language.

Publish the final QA recommendation and supporting artifacts.

28. Final Approval and Baseline Statement

This Test Plan establishes the approved portfolio baseline for manual functional testing of SauceDemo. Parts 1, 2, and 3 together define the project objectives, scope, test strategy, environment, data, coverage model, execution controls, defect process, risks, reporting, and closure criteria. Actual test outcomes will be recorded only in execution and closure artifacts.

Approval Role	Name	Status
QA Analyst / Test Owner	Nancy McKenzie	Approved for Portfolio
Business/Product	Simulated Stakeholder	Portfolio Simulation
Release Decision Owner	Simulated Product Owner	Decision recorded at test

29. Glossary

Term	Definition
AUT	Application Under Test. In this project, SauceDemo.
STLC	Software Testing Life Cycle: the structured activities used to plan, design, execute, report, and close testing.
RTM	Requirements Traceability Matrix used to map requirements to test coverage and results.
Defect	Observed behavior that differs from the defined or reasonably expected behavior.
Severity	Assessment of the technical/business impact of a defect.
Priority	Assessment of how urgently a defect should be addressed.
Regression Testing	Re-execution of relevant tests after change or correction to identify unintended impact.
Smoke Testing	Focused verification that critical application functions are available and stable enough for deeper testing.
Residual Risk	Known or possible quality risk remaining after planned testing and defect disposition.
Entry Criteria	Conditions that must be satisfied before meaningful test execution begins.
Exit Criteria	Conditions used to determine whether planned testing can close.
Blocked	A test cannot be completed because a prerequisite, environment, or application condition prevents execution.
Retest	Re-execution focused on confirming a specific defect correction.
Triage	Review process used to assess defect validity, impact, urgency, ownership, and next action.
Risk-Based Testing	Prioritization of testing according to likelihood and impact of failure.
Release Recommendation	QA assessment of whether available evidence supports release, release with known risk, or no release.